

Desarrollo de Sistemas Web

Clase 6

Docente: Delgado Joaquín



API y procesamiento de datos en E-commerce

- Promesas
- Manejo de Datos con Fetch y Promesas
- Manejo de Errores con Fetch
- Async/Await: Otra Forma de Esperar
- Buenas Prácticas de Accesibilidad y SEO
- Guía para el Proyecto Final de E-commerce

Promesas

Promesas en JavaScript

Cuando usás fetch, te devuelve una promesa: funciona como lo indica la palabra, es un compromiso del programa que tiene tres estados factibles: **pendiente**, **resuelta** o **rechazada**.

Resuelta (resolved)

La operación se completó con éxito.

Pendiente (pending)

La promesa está en proceso de resolución.

Rechazada (rejected)

La operación falló.

Manejo de datos con Fetch y promesas

Manejo de Datos con Fetch y Promesas

Imaginate que tu amiga te pasó la lista de compras para la cena, pero vos todavía no la miraste. Eso es lo que pasa cuando usás fetch: le pedís datos a una API y la promesa te los devuelve “a futuro”.

Ahora bien, cuando llega esa lista (los datos de la API), tenés que procesarla para ver qué cosas vas a comprar y cómo las vas a acomodar. En nuestro código, hacemos lo mismo:

En el siguiente ejemplo:

- Pedimos la lista de productos (como pedirle la lista a tu amiga).
- Esperamos que la promesa se cumpla (lleguen los datos).
- Procesamos esa lista para organizarla en el HTML (mostrando imagen, nombre, precio y botón de compra).
- Si falla, al menos nos enteramos con un `.catch()` (como si tu amiga nos dijera “*me olvidé la lista*”).

Ejemplo código:

```
fetch("https://fakestoreapi.com/products")
  .then((response) => response.json())
  .then((data) => {
    const contenedor = document.getElementById("productos-container");
    data.forEach((producto) => {
      contenedor.innerHTML += `
        <div class="card">
          
          <h3>${producto.title}</h3>
          <p>Precio: ${producto.price}</p>
          <button onclick="agregarAlCarrito(${producto.id})">Añadir al carrito</button>
        </div>
      `;
    });
  })
  .catch((error) => console.error("Error al obtener productos:", error));
```

Manejo de Errores con Fetch

Igual que en la vida, puede pasar que la tienda esté cerrada o que la lista esté incompleta. En programación, los errores también suceden, y hay que estar preparados.

Por ejemplo, si la respuesta no viene bien del servidor (códigos 400 o 500), podemos mostrarle un mensaje al usuario para que no se quede esperando:

1 Usar `.catch()`

Captura errores en la promesa de fetch.

2 Mostrar mensaje de error

Informa al usuario sobre el problema ocurrido.

Ejemplo:

```
fetch("https://fakestoreapi.com/products")
  .then((response) => {
    if (!response.ok) {
      throw new Error(`Error de servidor: ${response.status}`);
    }
    return response.json();
  })
  .then((data) => {
    // Mostrar productos normalmente
  })
  .catch((error) => {
    console.error("Error al obtener productos:", error);
    alert("Hubo un problema al cargar los productos. Por favor, intentá más tarde.");
  });
```

Explicación:

- Validamos si la respuesta es correcta (response.ok), porque a veces el servidor contesta “no encontré nada” (error 404) o “me rompí” (error 500).
- Si algo sale mal, usamos throw para “avisar fuerte” que no podemos seguir.
- Y con el .catch() le damos un mensaje al usuario, así no queda perdido sin explicación.

Async/Await

Async/Await: Otra Forma de Esperar

Hay otra forma de manejar la asincronía que es más sencilla y clara: **async/await**.

Async

Le decimos a la función que va a manejar tareas asíncronas.

¿Te acordás de la promesa de cenar con tu amiga? Con then le vas mandando mensajitos uno tras otro:

- *¿Podés venir?*
- *¿Traés algo?*
- *¿A qué hora llegás?*

Esos mensajes van encadenados, como pasa con las promesas en JavaScript.

Await

Le decimos que espere a que termine antes de seguir.

Pero con async/await, en vez de andar mandando mensajes sueltos, es como si se sentaran a charlar cara a cara y resuelven todo de una:

- *Confirmamos la cena.*
- *Definimos la hora.*
- *Ya sabemos qué traemos.*

Más directo, más claro, más fácil de leer.

Ejemplo con Async/Await

```
async function mostrarProductos() {  
  try {  
    const respuesta = await fetch("https://fakestoreapi.com/products");  
    const data = await respuesta.json();  
    // procesar datos...  
  } catch (error) {  
    console.error("Error al obtener productos:", error);  
  }  
}
```

Explicación:

- `async` indica que la función puede esperar tareas asíncronicas.
- `await` pausa el flujo hasta obtener la respuesta.

Así evitamos encadenar varios `.then`, y el código queda más limpio.

¿Por qué usar `async/await`?

- Porque hace el código más fácil de leer y entender ya que se parece a un paso a paso escrito en lenguaje natural en lugar de cadenas de `.then` evita anidaciones confusas y permite manejar errores con `try/catch` de forma más clara además la lectura es más lineal y sencilla de mantener

Buenas Prácticas de Accesibilidad y SEO

Buenas Prácticas de Accesibilidad



Etiquetas Alt

Usar atributos `alt` en imágenes para mejorar la experiencia de usuarios con discapacidades visuales.



Navegación por Teclado

Asegurar que los elementos interactivos sean accesibles mediante teclado.



Estructura Semántica

Utilizar etiquetas HTML semánticas para una mejor interpretación del contenido. (`<header>`, `<main>`, `<section>`, `<footer>`).

Optimización SEO en el Proyecto

1 Metaetiquetas

Las etiquetas `<meta>` en el `<head>` mejoran la visibilidad en buscadores.

2 Contenido Optimizado

Asegurar que las descripciones y nombres de productos sean claros y relevantes.

3 Encabezados Lógicos

Usar encabezados (`<h1>`, `<h2>`, `<h3>`) en una estructura jerárquica para que los motores de búsqueda puedan entender la relevancia del contenido.



Entrega de Proyecto Final

Proyecto Final de E-commerce

Desarrollarás una página web completa, que combine todos los conocimientos adquiridos sobre JavaScript. El proyecto consistirá en la creación de un sitio web de **e-commerce** dinámico e interactivo, que consuma datos de una API REST para mostrar productos, y permita a los usuarios añadir productos a un carrito de compras.

Pasos a seguir:

- **HTML:** Uso de etiquetas semánticas para organizar la página.
- **CSS:** Implementación de un diseño responsivo y atractivo usando Bootstrap y Flexbox.
- **Formulario funcional:** Crear un formulario de contacto con campos para **nombre**, **correo electrónico** y **mensaje**, utilizando **Formspree** para manejar el envío de datos.
- **JavaScript:** Integración de una API REST para obtener datos y renderizar productos en el DOM, además de la funcionalidad de un carrito de compras usando `localStorage`.
- **Accesibilidad y SEO:** Implementar prácticas que mejoren la experiencia del usuario y optimicen la página para los motores de búsqueda.«<

Formato de Entrega

1 Repositorio de GitHub

Crear un repositorio de GitHub (pública) que contenga los archivos y carpetas que conforman tu proyecto.

2 Hosting

Compartir el link de Github Pages o subida a servidor.

NOTA: crear un archivo **GrupoX.txt** (donde X es el número de grupo) con los siguientes datos:

- nombre de los integrantes.
- link al repositorio de GitHub.
- link de GitHub Pages.

Luego subir dicho archivo al siguiente repositorio:

https://drive.google.com/drive/folders/1TZvoCVMAX7M5wQhCiY7mZzJOhcdUX_Rq?usp=drive_link

🌟🌟 Funcionalidad esperada:

Interactividad Completa:

- La página debe permitir al usuario ver productos, añadirlos al carrito, editar el carrito, y simular la compra.

Formulario de Contacto:

- Implementa un formulario funcional que envíe datos a través de Formspree.

Diseño Responsivo:

- Asegúrate de que el diseño sea adaptable a diferentes tamaños de pantalla.

Persistencia del Carrito:

El carrito debe mantenerse activo incluso si el usuario cierra o actualiza la página, usando localStorage o sessionStorage.

¿Consultas o preguntas?



FIN